YEAR 12 · AI ENGINEERING · LECTURE 3

RAG

Retrieval-Augmented Generation: giving AI an open book

Don't make the model memorise everything — let it look things up.

TODAY

1. The closed-book problem
2. What RAG is
3. The architecture
4. Chunking and embeddings
5. Retrieve, augment, generate
6. RAG in LangChain
7. Making RAG better
8. Activity + quiz

The closed-book exam problem

- An LLM only knows what was in its **training data**, up to a **cut-off date**
- It has never seen **your** school handbook, timetable or class notes
- When it doesn't know, it may **make something up** confidently (hallucination)
- You usually can't check **where** an answer came from
- Retraining a model on new documents is **slow and expensive** — and has to be repeated whenever the documents change

TALKING POINTS

Ask the class: 'What is our school's policy on phones in class?' — then ask the same to a chatbot live if you can. It will either refuse or invent a plausible policy. That invented answer is the problem RAG solves. Use the exam analogy: the model is sitting a closed-book exam.

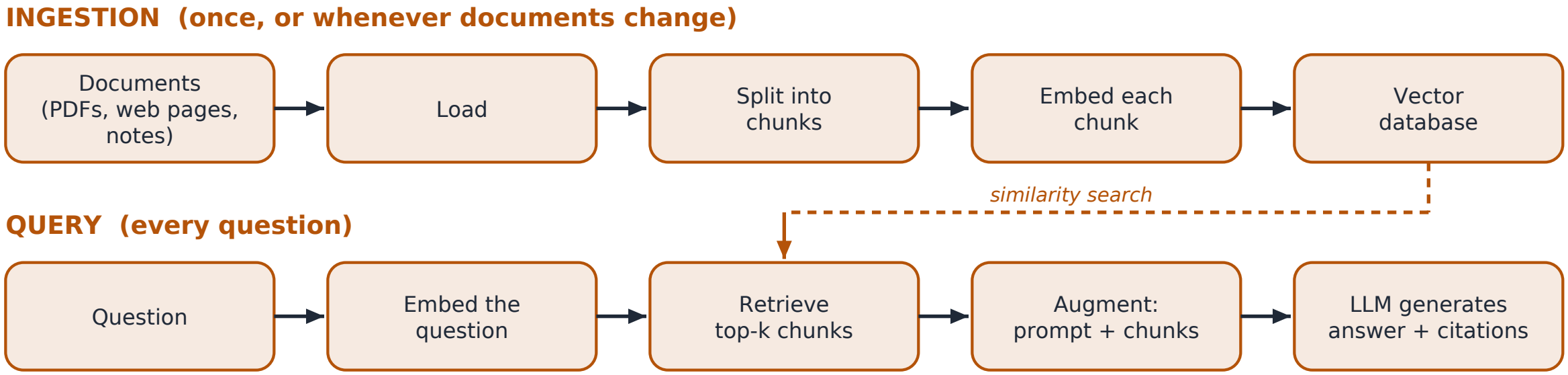
RAG = an open-book exam

- **R**etrieval — find the most relevant pieces of **your** documents
- **A**ugmented — paste them into the prompt as context
- **G**eneration — the LLM answers **using that context**, and can cite it
- Benefits:
  - **Up to date**: change the documents, not the model
  - **Private data** stays in your own database
  - **Fewer hallucinations** and answers you can **check** against the sources
  - **Cheap** compared with retraining or fine-tuning

TALKING POINTS

The open-book analogy carries the whole lecture: a student with the textbook open still has to find the right page (retrieval) and then write a good answer from it (generation). Both halves can go wrong, and we'll see how to fix each.

RAG architecture



TALKING POINTS

Two pipelines. Ingestion happens once (or whenever documents change): load, split into chunks, turn each chunk into an embedding, store in a vector database. The query pipeline runs for every question: embed the question, find the closest chunks, build a prompt with them, and generate an answer. Point at each box and ask students which analogy fits: chunks = index cards, vector DB = a library sorted by meaning.

Step 1 — Chunking: cutting documents into index cards

- Models can only read so much text at once, and smaller pieces make search **more precise**
- Split each document into **chunks** (e.g. 200–1,000 characters)
- Use **overlap** between chunks so an idea isn't cut in half
- Keep **metadata** with each chunk: file name, page, date, section
- Trade-off:
  - Chunks too **small** → missing context ("it is closed on Mondays" — what is?)
  - Chunks too **big** → irrelevant text, fewer chunks fit in the prompt

TALKING POINTS

Physical demo: take one page of the school handbook and cut it into strips. Ask a student to answer a question using only one strip — sometimes the strip is too small to make sense. That's the chunk-size trade-off. Metadata is what lets the bot say 'source: library.md, page 2'.

Step 2 — Embeddings and vector databases

- An **embedding** turns text into a list of numbers that captures its **meaning**
- Similar meanings → **nearby** vectors, even with different words:
  - "Phones must be kept in lockers" ≈ "Can I use my mobile in class?"
- **Cosine similarity** measures how close two vectors point (1 = same direction)
- A **vector database** stores millions of embeddings and finds the nearest ones fast
  - Examples: FAISS (library), Chroma, Qdrant, Pinecone, pgvector
- Keyword search (like TF-IDF) misses synonyms — embeddings don't

TALKING POINTS

Analogy: a normal library is sorted alphabetically; a vector database is a library sorted by meaning, so books about similar ideas sit on nearby shelves. The phone/mobile example is the one to remember: no shared words, but very similar meaning.

Step 3 — Augment: the RAG prompt

The retrieved chunks are pasted into a **prompt template** (Lecture 1!)

```
You are the school help desk. Answer the
question using ONLY the sources below.
Cite sources like [1]. If the sources don't
contain the answer, say "I don't know -
please ask the office."

Sources:
[1] (devices.md) Phones must be switched off
    and stored in lockers during class.
[2] (devices.md) Phones may be used at interval
    and lunch in outdoor areas only.
[3] (library.md) The library is open 8:00am
    to 4:30pm on weekdays.

Question: Can I use my mobile at lunchtime?
Answer:
```

Three instructions do the heavy lifting:

1. **ONLY** use the sources

2. **Cite** them

3. A safe answer when the information **isn't there**

TALKING POINTS

Read the prompt aloud. Ask what the model should answer (yes, outdoors only, citing [2]). Then ask: what if the question is 'Who is the principal?' The 'I don't know' instruction is what stops hallucination — without it, the model fills the gap with a guess.

RAG in LangChain


```
from langchain_community.document_loaders import DirectoryLoader, TextLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_community.vectorstores import FAISS
from langchain_core.runnables import RunnablePassthrough

docs = DirectoryLoader("handbook", glob="*.md", loader_cls=TextLoader).load()      # load
chunks = RecursiveCharacterTextSplitter(chunk_size=300, chunk_overlap=50).split_documents(docs)
store = FAISS.from_documents(chunks, HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2"))
retriever = store.as_retriever(search_kwargs={"k": 3})                          # top-3

def format_docs(ds):
    return "\n".join(f"[{i+1}] ({d.metadata['source']}) {d.page_content}" for i, d in enumerate(ds))

prompt = ChatPromptTemplate.from_template(
    "Answer using ONLY these sources and cite them like [1]. "
    "If unsure, say you don't know.\n\n{context}\n\nQuestion: {question}")
rag = ({"context": retriever | format_docs, "question": RunnablePassthrough()}
       | prompt | llm | StrOutputParser())
print(rag.invoke("How many books can I borrow and for how long?"))
```


TALKING POINTS

Map each line back to the architecture slide: load, split, embed + store, retriever, prompt, model, parser. Notice the whole query pipeline is one LCEL chain from Lecture 1. The dictionary at the start runs the retriever and passes the question through at the same time.

How do we know it works? Evaluate both halves

Check	Question it answers	How
Retrieval hit rate	Did the right chunk appear in the top-k?	A test set of questions + the source each should find
Faithfulness	Is every claim supported by the retrieved sources?	Human marking or an LLM judge
Answer correctness	Is the final answer right?	Compare with expected answers
Refusal	Does it say "I don't know" when it should?	Include questions that aren't in the documents

TALKING POINTS

If retrieval fails, even a perfect model can't answer — so always measure retrieval separately. Make sure students include 'trick' questions whose answers aren't in the documents; a good RAG bot must refuse those.

Making RAG better

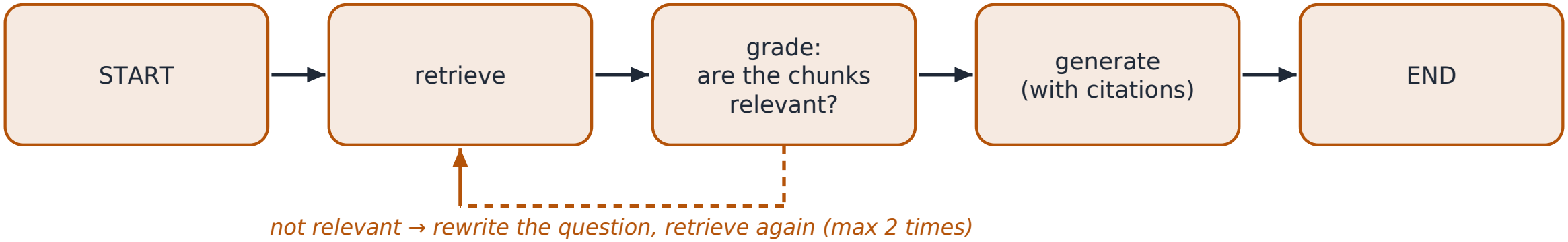
Technique	Idea
Better chunking	Split by headings/paragraphs; tune size and overlap
Hybrid search	Combine keyword search (exact names, codes) with embedding search (meaning)
Re-ranking	Retrieve 20 candidates fast, then a slower, smarter model re-orders them
Query rewriting	Rephrase vague questions before searching ("it" → "the library")
Metadata filters	Only search Year 12 documents, or this year's notices
Self-correcting RAG	A LangGraph loop: retrieve → grade the chunks → rewrite and retry if they're not relevant → generate

TALKING POINTS

Connect the last row to Lecture 2: that's a graph with a conditional edge and a loop limit. Emphasise 'measure, don't guess' — a technique that helps one dataset can hurt another.

Self-correcting RAG with LangGraph

Plain RAG retrieves **once** and hopes. A graph can **check its own retrieval**.



TALKING POINTS

Walk through: the grade node asks the LLM 'do these chunks answer the question? YES/NO'. If NO, a rewrite node rephrases the search query and we loop back. After two failed tries we generate 'I couldn't find that'. This is the bridge to agentic RAG, where an agent decides when and where to search.

RAG or fine-tuning?

You want the model to…	Best tool
Answer from **your documents**, which change over time	**RAG**
Cite sources so people can check	**RAG**
Write in a particular **style** or follow a fixed format	Fine-tuning (or a good system prompt)
Learn a new **skill** from many examples	Fine-tuning
Do something **once**	Just prompt it

TALKING POINTS

A common misconception is that you 'teach facts' by fine-tuning. Fine-tuning is unreliable for facts and hard to update; RAG is the standard answer for knowledge.

Activity — RAG detectives (12 minutes)

- Each group gets 6 short 'handbook' paragraphs (printed) and 5 questions
- **Chunk**: cut the paragraphs into chunks — decide the size
- **Retrieve**: for each question, pick the top-2 chunks (you are the vector database!)
- **Generate**: write the answer using only those chunks, with a citation
- One question has **no answer** in the handbook — did your group refuse correctly?
- Discuss: which question was hardest to retrieve for, and why?

TALKING POINTS

Unplugged activity that makes every step concrete. Include one question using different words from the text (mobile vs phone) and one with no answer. Debrief: humans also struggle when chunks are too small or the wording differs.

Recap + exit quiz

- **RAG** = retrieve relevant chunks → put them in the prompt → generate an answer with citations
- **Ingestion**: load → chunk → embed → store. **Query**: embed → retrieve → augment → generate
- Embeddings capture **meaning**; vector databases find the nearest chunks fast
- Evaluate **retrieval** and **answers** separately; teach the bot to say "I don't know"
- **Quiz:** 1) What does each letter of RAG stand for? 2) Why use chunk overlap? 3) Why can embedding search find "mobile" when the text says "phone"? 4) RAG or fine-tuning for a school-rules bot?

TALKING POINTS

Answers: 1) Retrieval, Augmented, Generation. 2) So ideas at chunk edges aren't cut in half. 3) Embeddings capture meaning, not exact words. 4) RAG — the rules change and answers should cite the handbook. Next: Lecture 4, keeping a human in the loop.